Laboratório 12

MQTT e RabbitMQ: Brokers, QoS e Tópicos

Tecnologias de Comunicação para a Internet das Coisas | Aula 12

Objetivo e enquadramento

A Aula 12 apresenta o paradigma orientado a mensagens, com o MQTT e o AMQP. Este laboratório instala dois brokers, o Mosquitto (MQTT) e o RabbitMQ (AMQP), numa máquina virtual, e torna observáveis o desacoplamento, os níveis de QoS, as mensagens retidas, os tópicos e as trocas de uma fila e de uma exchange.

O trabalho não requer acesso ao servidor da unidade curricular. Destina-se a ser realizado durante a aula, ao ritmo de cada aluno, e **não constitui trabalho a entregar**.

Nota

Os resultados apresentados são exemplos: identificadores, tempos e números de versão diferem de computador para computador.

Resultados de aprendizagem

- Publicar e subscrever mensagens MQTT, com tópicos hierárquicos e caracteres universais.
- Distinguir os níveis de QoS 0, 1 e 2 pelas trocas observadas.
- Demonstrar uma mensagem retida e um testamento (*last will*).
- Escrever um produtor e um consumidor com bibliotecas Python.
- Distinguir o modelo de fila do modelo de exchange de tópicos no RabbitMQ.

Requisitos

- Computador com Windows, macOS ou Linux, com cerca de 2 GB de memória livre.
- Ligação à Internet, necessária para obter a imagem e os pacotes.
- Duração estimada: entre 75 e 100 minutos.

Como utilizar este guião

Cada passo apresenta, por esta ordem, o objetivo, o enquadramento, o fundamento teórico, os comandos a executar e o resultado esperado. Recomenda-se ler o passo completo antes de executar o primeiro comando. As caixas têm sempre o mesmo significado:

Comando a executar

```
Comando a executar no terminal.
```

Resultado esperado (os valores podem variar)

```
Resultado que deve ser apresentado.
```

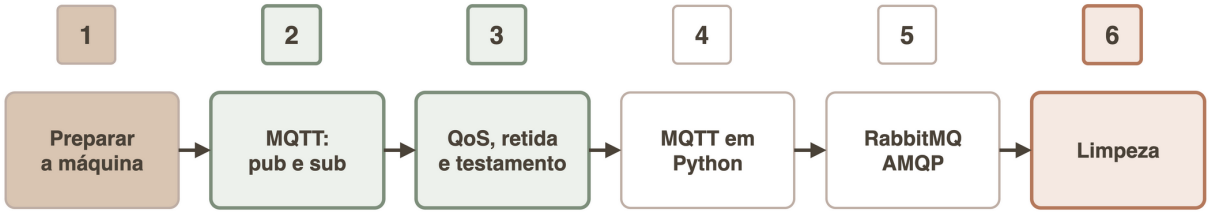
Analisar

Momento de pausa para interpretar o resultado obtido, isoladamente ou em discussão com os colegas.

Erro possível e resolução

Erro que pode ocorrer e respetiva resolução.

Percurso

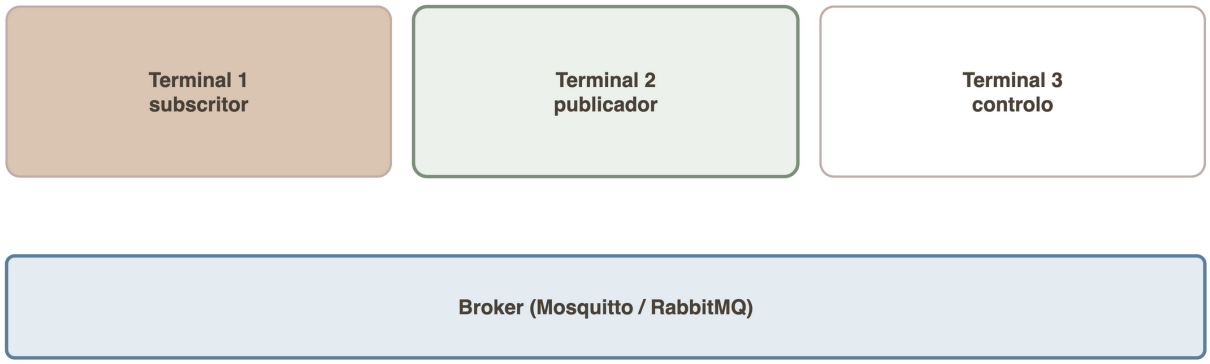


Dois brokers na mesma máquina: Mosquitto (MQTT) e RabbitMQ (AMQP)

Como ler a figura

Os seis passos leem-se da esquerda para a direita. O primeiro prepara a máquina. Os passos 2 e 3, a verde, exploram o MQTT com as ferramentas de linha de comandos. O passo 4 repete a ideia em Python e o passo 5 passa ao AMQP com o RabbitMQ. O passo 6 encerra o trabalho.

Em vários passos são necessários três terminais, todos ligados à mesma máquina virtual com `multipass shell tciot-mom`.



Os três terminais ligam-se à mesma VM: multipass shell tciot-mom

Como ler a figura

O terminal 1, a bege, fica a subscrever e deve permanecer aberto. O terminal 2, a verde, publica. O terminal 3, a branco, serve para comandos de controlo, como terminar processos. Em baixo está o broker, comum a todos.

Passo 1 — Preparação da máquina

Objetivo. Criar uma máquina virtual Ubuntu e instalar os dois brokers e as bibliotecas Python.

Fundamento teórico

O Mosquitto é um broker MQTT leve, e o RabbitMQ é um broker AMQP escrito em Erlang. Ambos correm como serviços do sistema e escutam em portos conhecidos: 1883 para MQTT e 5672 para AMQP. Por omissão, ambos aceitam apenas ligações locais, o que é adequado a este laboratório.

1.1 Criação da máquina e instalação

Comando a executar

```
multipass launch 22.04 --name tciot-mom --cpus 2 --memory 2G --disk 8G
multipass shell tciot-mom
```

Comando a executar

```
sudo apt update
sudo apt install -y mosquitto mosquitto-clients rabbitmq-server python3-venv
python3 -m venv ~/mq
~/mq/bin/pip install paho-mqtt pika
```

Comando a executar

```
systemctl is-active mosquitto rabbitmq-server
```

Resultado esperado (os valores podem variar)

```
active
active
```

Erro possível e resolução

launch failed: falta de memória livre. Fechar outras aplicações e repetir. Se a máquina já existir, `multipass delete tciot-mom`, `multipass purge` e repetir.

inactive no RabbitMQ: o arranque demora cerca de meio minuto. Aguardar e repetir o comando.

1.2 Os programas Python

Criar, na pasta pessoal, os dois ficheiros seguintes. Estão também na pasta `codigo` do material da aula.

Conteúdo do ficheiro `mqtt_demo.py`

```
"""MQTT demo (Lecture 12): publisher and subscriber with paho-mqtt.

Run in two terminals:
  python mqtt_demo.py sub          # subscribes to casa/+/temperatura
  python mqtt_demo.py pub [qos]    # publishes 5 readings
"""
import json
import random
import sys
import time

import paho.mqtt.client as mqtt

BROKER = "localhost"
TOPIC = "casa/sala/temperatura"
FILTER = "casa/+/temperatura"


def client():
    return mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)


def sub():
    def on_message(c, userdata, msg):
        print(f"{msg.topic} qos={msg.qos} retained={msg.retain} {msg.payload.
    decode()}")

    c = client()
    c.on_message = on_message
    c.connect(BROKER)
    c.subscribe(FILTER, qos=1)
    c.loop_forever()


def pub(qos=1):
    c = client()
```

```
    c.connect(BROKER)
    c.loop_start()
    for n in range(5):
        value = round(20 + random.random() * 3, 1)
        info = c.publish(TOPIC, json.dumps({"n": n, "value": value}), qos=qos)
        info.wait_for_publish()
        print(f"published n={n} value={value} qos={qos}")
        time.sleep(1)
    c.loop_stop()
    c.disconnect()


if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "sub":
        sub()
    else:
        pub(int(sys.argv[2]) if len(sys.argv) > 2 else 1)
```

Conteúdo do ficheiro `amqp_demo.py`

```
"""AMQP demo (Lecture 12): work queue and topic exchange with pika and RabbitMQ.

Terminals:
  python amqp_demo.py worker NAME        # competing consumer on the queue '
    tarefas'
  python amqp_demo.py send N             # sends N tasks to the queue
  python amqp_demo.py listen PATTERN     # binds a private queue to the exchange '
    casa' (topic)
  python amqp_demo.py emit KEY TEXT      # publishes TEXT with routing key KEY on
    'casa'
"""
import sys

import pika

HOST = "localhost"


def channel():
    conn = pika.BlockingConnection(pika.ConnectionParameters(HOST))
    return conn, conn.channel()


def worker(name):
    conn, ch = channel()
    ch.queue_declare(queue="tarefas", durable=True)
```

```
    ch.basic_qos(prefetch_count=1)

    def on_msg(c, method, props, body):
        print(f"[{name}] processing {body.decode()}")
        c.basic_ack(delivery_tag=method.delivery_tag)

    ch.basic_consume(queue="tarefas", on_message_callback=on_msg)
    ch.start_consuming()


def send(n):
    conn, ch = channel()
    ch.queue_declare(queue="tarefas", durable=True)
    for i in range(1, n + 1):
        ch.basic_publish("", "tarefas", f"task {i}".encode(),
                         pika.BasicProperties(delivery_mode=pika.DeliveryMode.
    Persistent))
        print(f"sent task {i}")
    conn.close()


def listen(pattern):
    conn, ch = channel()
    ch.exchange_declare(exchange="casa", exchange_type="topic")
    q = ch.queue_declare(queue="", exclusive=True).method.queue
    ch.queue_bind(exchange="casa", queue=q, routing_key=pattern)

    def on_msg(c, method, props, body):
        print(f"{method.routing_key}: {body.decode()}")

    ch.basic_consume(queue=q, on_message_callback=on_msg, auto_ack=True)
    ch.start_consuming()


def emit(key, text):
    conn, ch = channel()
    ch.exchange_declare(exchange="casa", exchange_type="topic")
    ch.basic_publish("casa", key, text.encode())
    print(f"published {key}: {text}")
    conn.close()


if __name__ == "__main__":
    cmd = sys.argv[1]
    {"worker": lambda: worker(sys.argv[2]), "send": lambda: send(int(sys.argv[2]))
    ,
     "listen": lambda: listen(sys.argv[2]), "emit": lambda: emit(sys.argv[2], sys.
    argv[3])}[cmd]()
```

Passo 2 — MQTT com publicador e subscritor

Objetivo. Publicar e subscrever mensagens por tópico e observar o desacoplamento.

Fundamento teórico

Num broker MQTT, os tópicos são cadeias hierárquicas separadas por barras. Um subscritor indica um filtro, que pode conter o sinal `+`, que substitui um nível, e o cardinal `#`, que substitui todos os níveis abaixo. O publicador não conhece os subscritores: publica no tópico e o broker entrega a quem tem filtro compatível.

A analisar neste passo: o publicador e o subscritor não trocam endereços nem estão ligados entre si.

No terminal 1, subscrever tudo o que está abaixo de `casa`:

Comando a executar

```
mosquitto_sub -h localhost -t 'casa/#' -v
```

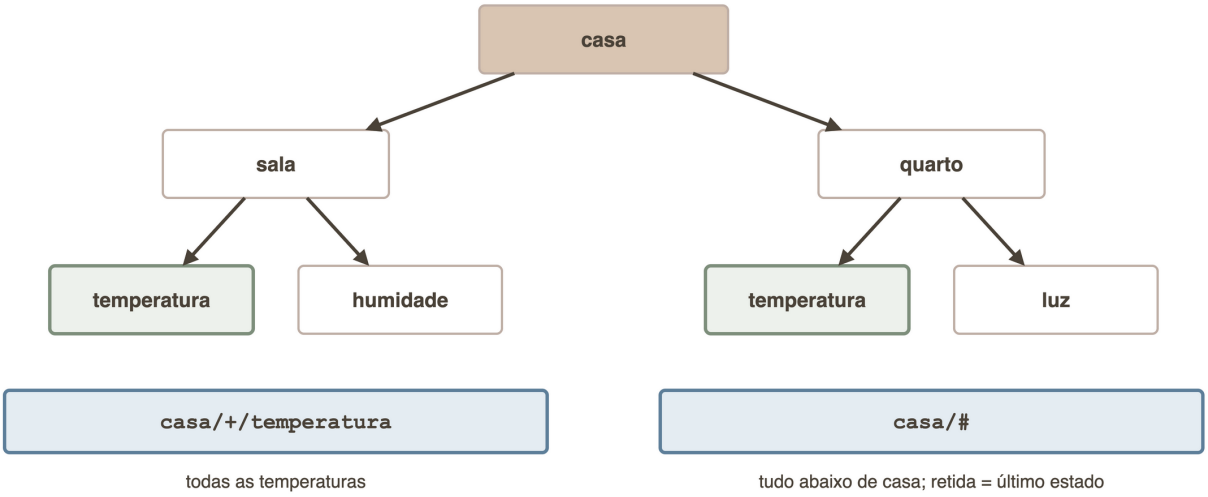
No terminal 2, publicar duas mensagens:

Comando a executar

```
mosquitto_pub -h localhost -t casa/sala/temperatura -m 21.5
mosquitto_pub -h localhost -t casa/quarto/luz -m on
```

Resultado esperado (os valores podem variar)

```
casa/sala/temperatura 21.5
casa/quarto/luz on
```



Como ler a figura

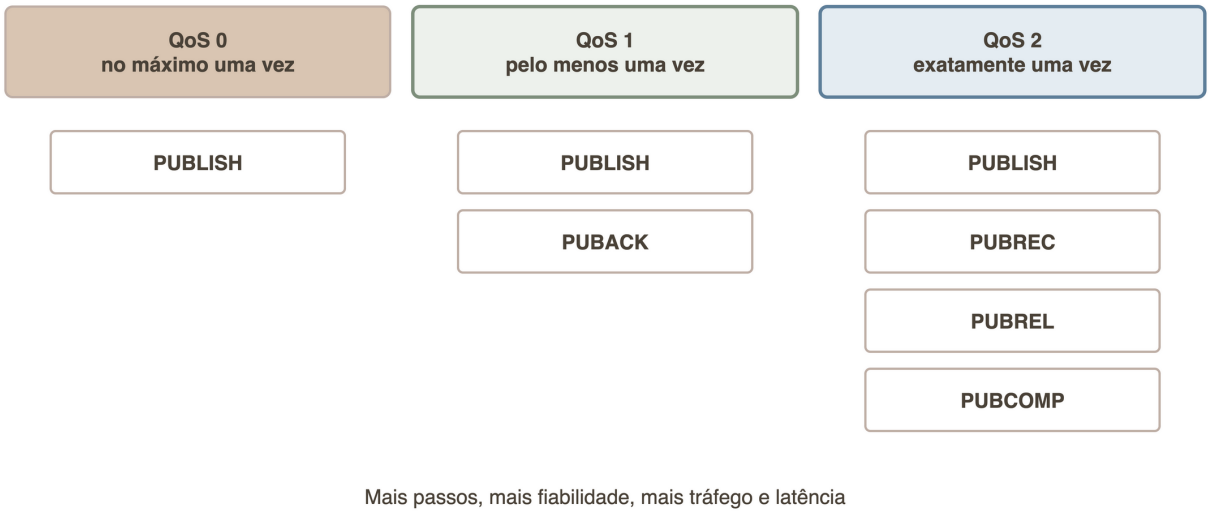
A figura mostra a árvore de tópicos. A subscrição `casa/#` apanha tudo, e `casa/+/temperatura` apanha as temperaturas de qualquer divisão. As duas mensagens publicadas acima são apanhadas pela primeira.

Analisar

Termine o subscritor e volte a subscrever com `casa/+/temperatura`. Que mensagens recebe agora? O que aconteceu à mensagem da luz?

Passo 3 — QoS, mensagens retidas e testamento

Objetivo. Observar as trocas de cada nível de QoS, o efeito de uma mensagem retida e o testamento de um cliente.



Como ler a figura

Cada coluna é um nível de QoS e cada caixa uma mensagem trocada. O nível 0 tem uma só mensagem, o nível 1 acrescenta uma confirmação e o nível 2 usa quatro mensagens.

Fundamento teórico

O QoS é pedido por quem publica e por quem subscreve, e o nível efetivo é o menor dos dois. Uma mensagem retida é guardada pelo broker e entregue a quem subscrever depois. O testamento é uma mensagem que o cliente entrega ao broker quando se liga, e que este publica se a ligação terminar sem um DISCONNECT normal.

3.1 Trocas de cada QoS

Publicar com `-d`, que apresenta os pacotes de protocolo:

Comando a executar

```
mosquitto_pub -d -q 0 -t casa/sala/temperatura -m 21.5
mosquitto_pub -d -q 1 -t casa/sala/temperatura -m 21.5
mosquitto_pub -d -q 2 -t casa/sala/temperatura -m 21.5
```

Resultado esperado (os valores podem variar)

```
(q0) sending PUBLISH (d0, q0, r0, m1, 'casa/sala/temperatura', ... (4 bytes))
(q1) sending PUBLISH (d0, q1, r0, m1, ...)  received PUBACK (Mid: 1, RC:0)
(q2) sending PUBLISH (d0, q2, r0, m1, ...)  received PUBREC (Mid: 1, RC:0)
     sending PUBREL (Mid: 1)  received PUBCOMP (Mid: 1, RC:0)
```

3.2 Mensagem retida

Comando a executar

```
mosquitto_pub -r -t casa/sala/temperatura -m 21.5
mosquitto_sub -t casa/sala/temperatura -C 1 -v
```

Resultado esperado (os valores podem variar)

```
casa/sala/temperatura 21.5
```

O segundo comando termina de imediato, porque o broker entrega a mensagem retida logo que a subscrição é feita. Para a apagar: `mosquitto_pub -r -n -t casa/sala/temperatura`.

3.3 Testamento

No terminal 1, subscrever o estado:

Comando a executar

```
mosquitto_sub -t estado/sensor1 -v
```

No terminal 2, ligar um cliente com testamento:

Comando a executar

```
mosquitto_sub -t qualquer --will-topic estado/sensor1 --will-payload offline
```

No terminal 3, terminar o cliente de forma abrupta:

Comando a executar

```
pkill -9 -f 'will-topic'
```

Resultado esperado (os valores podem variar)

```
estado/sensor1 offline
```

Analisar

Quem publicou a mensagem `offline`? Porque é que a publicação só acontece quando a ligação termina de forma anormal? Como se usaria isto, em combinação com mensagens retidas, para representar a presença de um sensor?

Passo 4 — MQTT em Python

Objetivo. Escrever um produtor e um consumidor com a biblioteca `paho-mqtt`.

Fundamento teórico

Uma aplicação MQTT liga-se ao broker, subscreve filtros e trata cada mensagem numa função de retorno. O produtor publica com um QoS à escolha e, no QoS 1, aguarda a confirmação com `wait_for_publish`. O consumidor do exemplo subscreve `casa/+/temperatura` com QoS 1 e imprime o tópico, o QoS e o indicador de mensagem retida.

No terminal 1:

Comando a executar

```
~/mq/bin/python mqtt_demo.py sub
```

No terminal 2:

Comando a executar

```
~/mq/bin/python mqtt_demo.py pub 1
```

Resultado esperado (os valores podem variar)

```
published n=0 value=22.3 qos=1
published n=1 value=21.1 qos=1
...
```

Resultado esperado (os valores podem variar)

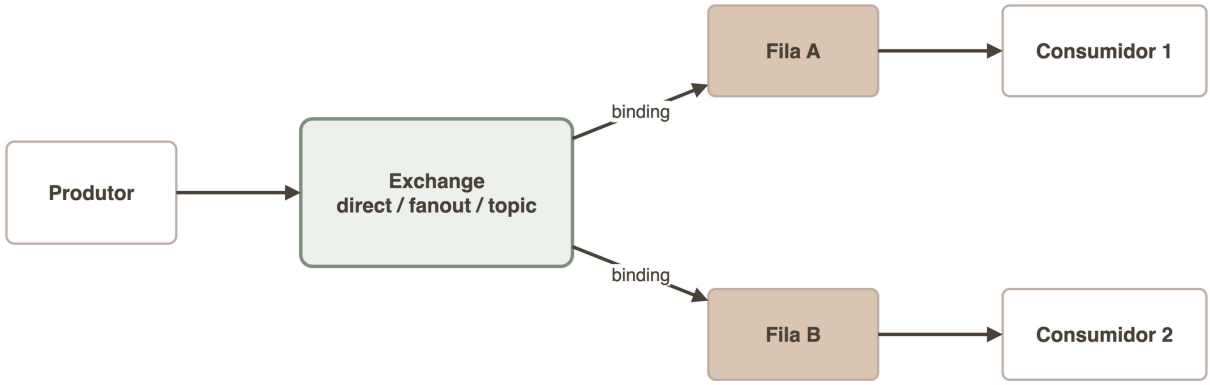
```
casa/sala/temperatura qos=1 retained=False {"n": 0, "value": 22.3}
casa/sala/temperatura qos=1 retained=False {"n": 1, "value": 21.1}
```

Analisar

Altere o QoS do publicador para 0 e repita. Que diferença nota no consumidor? Quantos consumidores podem ser acrescentados sem alterar o produtor?

Passo 5 — AMQP com o RabbitMQ

Objetivo. Observar uma fila com consumidores concorrentes e uma exchange de tópicos.



A exchange decide, pelo tipo e pelas ligações, para que filas segue a mensagem

Como ler a figura

A mensagem do produtor chega à exchange, que a encaminha para as filas segundo as ligações. Em cada fila, as mensagens são entregues aos consumidores por ordem. Na fila de tarefas, cada mensagem vai a um único consumidor.

Fundamento teórico

Numa fila partilhada, o RabbitMQ distribui as mensagens pelos consumidores por ordem, e com `prefetch_count=1` dá uma nova mensagem apenas a quem terminou a anterior. Numa exchange do tipo *topic*, a chave de encaminhamento é uma sequência de palavras separadas por pontos, e as ligações usam `*`, que substitui uma palavra, e `#`, que substitui zero ou mais. Note que o separador é o ponto, ao contrário do MQTT.

5.1 Fila de tarefas

Nos terminais 1 e 2, lançar dois trabalhadores:

Comando a executar

```
~/mq/bin/python amqp_demo.py worker A
~/mq/bin/python amqp_demo.py worker B
```

No terminal 3, enviar seis tarefas:

Comando a executar

```
~/mq/bin/python amqp_demo.py send 6
```

Resultado esperado (os valores podem variar)

```
[A] processing task 1
[B] processing task 2
[A] processing task 3
[B] processing task 4
[A] processing task 5
[B] processing task 6
```

5.2 Exchange de tópicos

Terminar os trabalhadores. Nos terminais 1 e 2, ligar dois ouvintes com padrões diferentes:

Comando a executar

```
~/mq/bin/python amqp_demo.py listen 'casa.*.temperatura'
~/mq/bin/python amqp_demo.py listen 'casa.#'
```

No terminal 3:

Comando a executar

```
~/mq/bin/python amqp_demo.py emit casa.sala.temperatura 21.5
~/mq/bin/python amqp_demo.py emit casa.quarto.luz on
```

Resultado esperado (os valores podem variar)

```
casa.sala.temperatura: 21.5           (listener 1 and listener 2)
casa.quarto.luz: on                     (listener 2 only)
```

5.3 Interface de gestão

Comando a executar

```
sudo rabbitmq-plugins enable rabbitmq_management
curl -s -u guest:guest http://localhost:15672/api/overview | head -c 200
sudo rabbitmqctl list_queues name messages consumers
```

Analisar

Quantos consumidores tem a fila de tarefas depois de terminar os trabalhadores? O que aconteceria às mensagens enviadas enquanto nenhum trabalhador estava ligado, e porquê?

Erro possível e resolução

Connection refused no RabbitMQ: o serviço ainda está a arrancar. `sudo systemctl status rabbitmq-server` e aguardar. O utilizador `guest` só pode ligar-se a partir da própria máquina.

Passo 6 — Limpeza

Comando a executar

```
exit
multipass delete tciot-mom
multipass purge
```

Questões de consolidação

1. Porque é que o publicador e o subscritor MQTT não precisam de se conhecer?
2. Que diferença há entre QoS 1 e QoS 2 nas mensagens trocadas?
3. Que utilidade tem o testamento num sensor alimentado a bateria?
4. Qual é a diferença entre dois consumidores na mesma fila e dois ouvintes ligados a uma exchange de tópicos?
5. Que separador usam os tópicos MQTT e que separador usam as chaves de encaminhamento AMQP?

Síntese

- O **broker** desacopla produtores e consumidores no espaço, no tempo e na sincronização.
- O **QoS** do MQTT escolhe-se por mensagem e muda o número de trocas.
- As **mensagens retidas** e o **testamento** dão estado e presença a quem chega depois.
- No **AMQP**, a exchange e as ligações decidem para onde a mensagem vai, e as filas repartem o trabalho.